

Бинарные файлы (Файлы. Запись. Класс BinaryWriter и BinaryReader)

BinaryWriter – Класс для записи типизированных данных в поток (FileStream).

Методы записи:

Write(bool) - логическое значение

Write(byte) - один байт

Write(int) - целое число (4 байта)

Write(double) - число с плавающей запятой (8 байт)

Write(string) - строка (с указанием длины в начале)

Write(char) - один символ

FileStream

Используется вместе с конструктором **FileStream**.

FileStream fs = new FileStream(путь, режим, доступ);

Режимы (FileMode):

Create - создать новый файл (перезапишет, если существует)

Open - открыть существующий файл

OpenOrCreate - открыть или создать

Append - открыть для добавления в конец

Доступ (FileAccess):

Read - только чтение

Write - только запись

ReadWrite - чтение и запись

Пример использования:

```
internal class Program
{
    static void Main(string[] args)
    {
        using (FileStream fs = new FileStream("data.bin", FileMode.Create))
        {
            BinaryWriter writer = new BinaryWriter(fs);
            writer.Write(25);           // записываем число
            writer.Write("Иван");       // записываем строку
            writer.Write(3.14);         // записываем double
            writer.Close();
        }
    }
}
```

BinaryReader – Класс для чтения типизированных данных из потока (FileStream).

Методы чтения:

ReadBoolean() - логическое значение

ReadByte() - один байт

ReadInt32() - целое число

ReadDouble() - число с плавающей запятой

ReadString() - строка

ReadChar() - один символ

Пример использования:

```
using (FileStream fs = new FileStream("data.bin", FileMode.Open))
{
    BinaryReader reader = new BinaryReader(fs);
    int age = reader.ReadInt32();
    string name = reader.ReadString();
    double price = reader.ReadDouble();
    reader.Close();
}
```

Что такое LINQ?

LINQ (Language Integrated Query - интегрированный в язык запрос) – это мощная технология в C# для работы с коллекциями данных. LINQ позволяет писать запросы к данным на удобном и понятном синтаксисе.

Пространство имен

```
using System.Linq; // обязательно для использования LINQ
```

Синтаксис LINQ

```
var result = students.Where(s => s.Age > 18)
    .OrderBy(s => s.Name)
    .Select(s => s.Name);
```

Основные методы LINQ

Where(predicate) - фильтрует элементы

OrderBy(selector) - сортирует по возрастанию

OrderByDescending(selector) - сортирует по убыванию

Select(selector) - преобразует элементы

Average(selector) - вычисляет среднее арифметическое

First(), Last() - первый/последний элемент

Count() - количество элементов

ToArray(), ToList() - преобразует в массив/список

Для решения задач нам нужны структуры (struct) или классы (class), которые будут содержать информацию об объектах.

Задача 1: Средний балл учеников

Задание: вычислить средний балл учеников класса. Распечатать список учеников, имеющих средний балл выше среднего в классе.

Что используем для выполнения задачи:

Array.Sort() из пространства System - сортирует массив

Array.Clear() - очищает массив

LINQ (Where, Average, OrderBy) - для фильтрации и сортировки

```
1  using System;
2  using System.IO;
3  using System.Collections.Generic;
4  using System.Linq;
5
6  public struct Student
7  {
8      public string Name;
9      public int MathGrade;
10     public int RussianGrade;
11     public int PhysicsGrade;
12
13     public double AverageGrade
14     {
15         get { return (MathGrade + RussianGrade + PhysicsGrade) / 3.0; }
16     }
17 }
18
19 class Program
20 {
21     // Метод для записи студентов в бинарный файл
22     static void SaveStudents(string filename, Student[] students)
23     {
24         using (FileStream fs = new FileStream(filename, FileMode.Create))
25         {
26             BinaryWriter writer = new BinaryWriter(fs);
27             writer.Write(students.Length);
28
29             foreach (var student in students)
30             {
31                 writer.Write(student.Name);
32                 writer.Write(student.MathGrade);
33                 writer.Write(student.RussianGrade);
34                 writer.Write(student.PhysicsGrade);
35             }
36             writer.Close();
37         }
38     }
39 }
```

```

40 // Метод для чтения студентов из бинарного файла
41 static Student[] StudentsFile(string filename)
42 {
43     using (FileStream fs = new FileStream(filename, FileMode.Open))
44     {
45         BinaryReader reader = new BinaryReader(fs);
46         int count = reader.ReadInt32();
47         Student[] students = new Student[count];
48
49         for (int i = 0; i < count; i++)
50         {
51             students[i].Name = reader.ReadString();
52             students[i].MathGrade = reader.ReadInt32();
53             students[i].RussianGrade = reader.ReadInt32();
54             students[i].PhysicsGrade = reader.ReadInt32();
55         }
56         reader.Close();
57         return students;
58     }
59 }

```

```

61 static void Main()
62 {
63     // Создаем массив студентов
64     Student[] students = new Student[]
65     {
66         new Student { Name = "Петров", MathGrade = 4, RussianGrade = 3, PhysicsGrade = 4 },
67         new Student { Name = "Сидоров", MathGrade = 5, RussianGrade = 5, PhysicsGrade = 4 },
68         new Student { Name = "Иванов", MathGrade = 3, RussianGrade = 4, PhysicsGrade = 3 },
69         new Student { Name = "Козлов", MathGrade = 5, RussianGrade = 5, PhysicsGrade = 5 },
70     };
71
72     // Сохраняем в файл
73     string filename = "students.bin";
74     SaveStudents(filename, students);
75     Console.WriteLine("Данные сохранены в файл");
76
77     // Загружаем из файла
78     students = StudentsFile(filename);
79
80     // Вычисляем средний балл в классе
81     double classAverage = students.Average(s => s.AverageGrade);
82     // AVERAGE() - метод LINQ, вычисляет среднее арифметическое
83
84     Console.WriteLine($"Средний балл в классе: {classAverage:F2}");
85     Console.WriteLine("\nУченики выше среднего балла класса:");
86
87     // Фильтруем студентов с баллом выше среднего
88     var aboveAverage = students.Where(s => s.AverageGrade > classAverage)
89                               .OrderByDescending(s => s.AverageGrade);
90     // WHERE() - фильтрует элементы
91     // ORDERBYDESCENDING() - сортирует в обратном порядке
92
93     foreach (var student in aboveAverage)
94     {
95         Console.WriteLine($"{student.Name}: {student.AverageGrade:F2}");
96     }
97 }
98 }

```

Задача 2: Фамилии рабочих

Задание: распечатать фамилии рабочих бригады, начинающиеся с букв А и С, с указанием их месячной зарплаты.

Что используем для выполнения задачи:

StartsWith() - проверяет, начинается ли строка с указанного символа

```
1  using System;
2  using System.IO;
3  using System.Collections.Generic;
4  using System.Linq;
5
6  public struct Worker
7  {
8      public string LastName;
9      public decimal MonthlySalary;
10 }
11
12 class Program
13 {
14     static void SaveFile(string filename, Worker[] workers)
15     {
16         using (FileStream fs = new FileStream(filename, FileMode.Create))
17         {
18             BinaryWriter writer = new BinaryWriter(fs);
19             writer.Write(workers.Length);
20
21             foreach (var worker in workers)
22             {
23                 writer.Write(worker.LastName);
24                 writer.Write(worker.MonthlySalary);
25             }
26             writer.Close();
27         }
28     }
29
30     static Worker[] LoadFile(string filename)
31     {
32         using (FileStream fs = new FileStream(filename, FileMode.Open))
33         {
34             BinaryReader reader = new BinaryReader(fs);
35             int count = reader.ReadInt32();
36             Worker[] workers = new Worker[count];
37
38             for (int i = 0; i < count; i++)
39             {
40                 workers[i].LastName = reader.ReadString();
41                 workers[i].MonthlySalary = reader.ReadDecimal();
42             }
43             reader.Close();
44             return workers;
45         }
46     }
47 }
```

```
48 static void Main()
49 {
50     Worker[] workers = new Worker[]
51     {
52         new Worker { LastName = "Алексеев", MonthlySalary = 35000 },
53         new Worker { LastName = "Борисов", MonthlySalary = 40000 },
54         new Worker { LastName = "Сергеев", MonthlySalary = 38000 },
55         new Worker { LastName = "Дмитриев", MonthlySalary = 42000 },
56         new Worker { LastName = "Александров", MonthlySalary = 36000 },
57     };
58
59     string filename = "workers.bin";
60     SaveFile(filename, workers);
61     Console.WriteLine("Данные рабочих сохранены\n");
62
63     workers = LoadFile(filename);
64
65     Console.WriteLine("Рабочие, начинающиеся на А и С:");
66
67     var filtered = workers.Where(w => w.LastName.StartsWith("А") ||
68                                     w.LastName.StartsWith("С"))
69                             .OrderBy(w => w.LastName);
70     // STARTSWITH() - проверяет начало строки
71
72     foreach (var worker in filtered)
73     {
74         Console.WriteLine($"{worker.LastName}: {worker.MonthlySalary} руб.");
75     }
76 }
77 }
```

Варианты выполнения 10.2 Файлы.Запись

1	Арютинa Анастасия
2	Аскарoв Арслан
3	Галеев Альмир
4	Гилемьянова Камилла
5	Гирфанов Байрас
6	Громыхалин Дмитрий
7	Зиангиров Камиль
8	Ибрагимов Ринат
9	Иванов Евгений
10	Казарбеков Ильяс
11	Каримов Карим
12	Киценко Данила
13	Кузнецов Кирилл
14	Нуретдинова Диля
15	Парфенов Александр
16	Порываева София
17	Сафин Салават
18	Саханевич Макар
19	Содиков Даниил
20	Тимербаева Элина
21	Уразов Руслан
22	Фазуллина Карина
23	Шагеев Артём
24	Шагимов Рахим